Programming Assignment 4

Start Assignment

- Due Monday by 11:59pm
- Points 100
- Submitting a text entry box or a website url
- Available Oct 13 at 12am - Nov 2 at 11:59pm

🎯 Word Guessing Game

Practice building a Python game while using Git, pytest, and GitHub for version control, testing, and project management.

🎯 Assignment Goal

Create a word-guessing game where the user attempts to guess a word one letter at a time. You will build the game in Python, create tests using **pytest**, and use **Git and GitHub** to track and share your work.

This exercise gives you an opportunity to practice not only programming concepts, but also the development workflow used to manage, test, and track changes to a software project.

📋 Part 1: GitHub Repository Setup

1. Log in to GitHub (https://github.com) or create a new account if you do not already have one.
2. Click **"+"** → **"New Repository."**
3. Name your repository `CIS256_Full_Name_PA4`.

4. Set the repository to **Public**, add a **README file**, and click **“Create repository.”**
5. Open Command Prompt on Windows or Terminal on Mac/Linux.
6. Navigate to the folder where you want your project to be created. For example:

```
cd Documents
```

   or

```
cd Desktop
```

7. Copy your repository’s HTTPS link from GitHub. It should look similar to:

```
https://github.com/yourusername/CIS256_Full_Name_PA4.git
```

8. Clone the repository to your computer:

```
git clone https://github.com/yourusername/CIS256_Full_Name_PA4.git
```

   Note: If you receive an error such as `'git' is not recognized as an internal or external command`, Git may not be installed on your computer. You can download Git from **Git Downloads (https://git-scm.com/downloads)**. After installing Git, close and reopen Command Prompt or Terminal and try the command again.

9. **Move into your repository folder:**

```
cd CIS256_Full_Name_PA4
```

🎮 Part 2: Write the Game

1. Open **IDLE**.
2. Create a new file using **File** → **New File** and save it in your cloned repository folder as `guess_the_word.py`.

3. **Implement the game:** Create a word-guessing game with the following features:
    - Select a random word from a predefined list.
    - Prompt the user to guess one letter at a time.
    - Reveal letters when the user guesses correctly and indicate when a guess is incorrect.
    - If a letter appears more than once in the word, reveal all instances of that letter when it is guessed correctly.
    - Allow letters to be guessed in any order and reveal correct guesses immediately.
    - Continue until the user guesses the word or runs out of attempts.
    - Display a congratulatory message when the word is successfully guessed.
4. **Include thoughtful comments:** Use comments throughout your code to explain important parts of your program.
5. **Save your changes frequently** in IDLE as you develop your program.

🧪 Part 3: Write Tests Using pytest

1. Create a new file in IDLE and save it in your cloned repository folder as `test_guess_the_word.py`.
2. **Write tests** that check:
    - The selected word comes from the predefined list.
    - Guesses are processed correctly, including correct and incorrect guesses.
3. **Include thoughtful comments:** Use comments throughout your test code to explain important parts of your tests.

Your tests should demonstrate that you understand how **pytest** can be used to verify that different parts of your program behave as expected.

🔄 Part 4: Use Git to Track Your Changes

Git should be used continuously throughout Parts 2 and 3 to track the development of your project.

After creating or updating your files, use the following commands from your project folder:

```
git add .
git commit -m "Added game logic and tests"
git push
```

- Replace `"Added game logic and tests"` with a meaningful message describing the change you made.
- Commit your work frequently as you develop your project.
- Make at least **3 commits outside of the original file save commits**, with meaningful commit messages that demonstrate progress in your Python files.

📋 Required Header

Add the following comments to the top of **both** Python files before submitting:

```
# Your Name
# CIS256 Term Year (e.g. Fall 2026)
# Programming Assignment 4
```

📤 What to Submit

Submit the **URL to your public GitHub repository**:

```
https://github.com/yourusername/CIS256_Full_Name_PA4
```

Before submitting, make sure your repository contains:

- `guess_the_word.py`
- `test_guess_the_word.py`
- A visible commit history showing your development progress.
- At least **3 meaningful commits** beyond the original file save commits.
- The required header comments in both Python files.

Programming Assignment & Git Hub Rubric - 100 Pts (1)

Criteria	Ratings				Pts
Submission Requirements GitHub link submitted using the required naming convention and all .py files include required header comments, appropriate PEP 8 formatting, and comments throughout the code where appropriate.	**10 pts** **Advanced** All submission requirements were completed correctly, including appropriate file naming, required header comments, appropriate PEP 8 formatting, and comments throughout the code where appropriate.	**8 pts** **Proficient** Most submission requirements were completed correctly, with only minor areas needing improvement, such as being slightly lacking in comments or having minor PEP 8 formatting issues.	**5 pts** **Needs Improvement** Some submission requirements were completed correctly, but the submission is lacking in comments, header information, or appropriate PEP 8 formatting.	**0 pts** **Incomplete** Submission requirements were largely not completed, including missing or incorrect file naming, header comments, comments throughout the code, or appropriate PEP 8 formatting.	10 pts
Program Requirements Program includes the required components and follows the specifications stated in the assignment.	**25 pts** **Advanced** All required program components and specifications are completed, with a clear and complete effort throughout the program.	**20 pts** **Proficient** Most required program components and specifications are effortfully completed, with only minor omissions or deviations.	**12 pts** **Needs Improvement** Some required program components or specifications are effortfully completed, but significant requirements are missing or incomplete.	**0 pts** **Incomplete** Few or none of the required program components or specifications are completed.	25 pts
Program Execution Program demonstrates an appropriate attempt to implement the	**20 pts** **Advanced** Program executes successfully without errors and demonstrates	**16 pts** **Proficient** Program executes successfully with minor errors or incomplete	**10 pts** **Needs Improvement** Program has significant execution errors or demonstrates a limited	**0 pts** **Incomplete** Program does not execute successfully and demonstrates little to no	20 pts

Criteria	Ratings				Pts
required program and executes successfully.	a complete and appropriate attempt to implement the required program.	elements and demonstrates an appropriate attempt to implement most of the required program.	attempt to implement the required program.	meaningful attempt to implement the required program.	
Programming Concepts Program correctly applies the Python programming concepts being assessed and demonstrates understanding of those concepts.	**25 pts** **Advanced** Program correctly implements all required programming concepts. Code demonstrates a strong understanding of the concepts being assessed.	**20 pts** **Proficient** Program implements most required programming concepts correctly, with only minor errors or misunderstandings.	**12 pts** **Needs Improvement** Program implements some required programming concepts, but significant errors or misunderstandings are present.	**0 pts** **Incomplete** Program does not demonstrate the required programming concepts or demonstrates little understanding of the concepts being assessed.	25 pts
GitHub Requirements Required non-code activities and components are completed according to the specifications stated in the exercise.	**20 pts** **Advanced** All required GitHub components are completed thoroughly and appropriately.	**16 pts** **Proficient** Most required GitHub components are completed, with only minor omissions or incomplete portions.	**10 pts** **Needs Improvement** Some required GitHub components are completed, but several are missing or incomplete.	**0 pts** **Incomplete** Few or none of the required non-code activities or components are completed.	20 pts
					Total Points: 100